

partnr Dashboard

Skill
Graph

Profile

GPP

Job
Applications

Maneesh Koti B... ▾

Build an iNaturalist Species Image Classifier with PyTorch and Grad-CAM

[Back](#)

Mandatory

Domain

Data Science

Machine Learning

Skills

API

Deep Learning

Machine Learning

Model Evaluation

Model Interpretability

Public Relations

Difficulty

Intermediate

Tools

Docker

Matplotlib

Numpy

Pandas

Pillow

Python

Pytorch



Pending Submission

TIME REMAINING

Please submit your work when ready.

5dDeadline: **3 Oct 2026 at 04:59 PM**

Overview

Instructions

Resources

Submit

Objective

Build an end-to-end image classification pipeline using the iNaturalist dataset. You will learn to acquire data from a public API, implement transfer learning by fine-tuning a MobileNetV3 model with PyTorch, and interpret model predictions using Grad-CAM. These skills are fundamental for developing and deploying trustworthy computer vision solutions in real-world applications like biodiversity monitoring.

Description

Introduction

Welcome to the iNaturalist Species Image Classification challenge. In this project, you will build a complete machine learning pipeline to identify wildlife species from images. You'll use the vast iNaturalist open dataset, which contains millions of crowd-sourced observations of plants and animals. This task mirrors the work of data scientists in conservation tech and ecological research, where automated species identification is crucial for monitoring biodiversity at scale.

Your goal is to fetch data for a specific group of animals, fine-tune a modern, efficient computer vision model, evaluate its performance, and, most importantly, interpret its decisions. This end-to-end process will solidify your skills in data engineering, model training, and the emerging field of explainable AI (XAI).

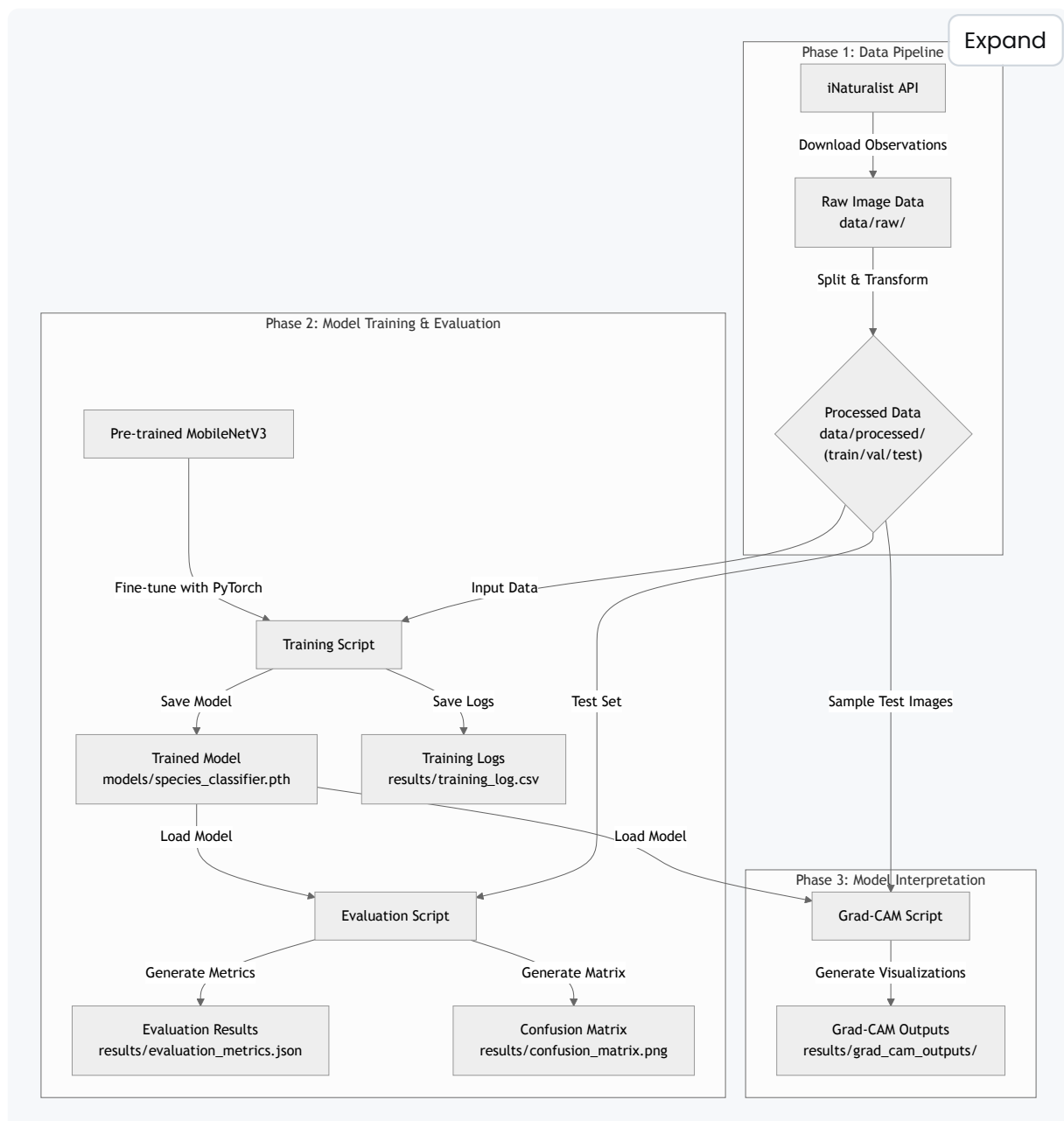
Core Technologies and Concepts

1. **iNaturalist API:** This is your data source. iNaturalist provides a free, public API to access its rich dataset of geolocated, time-stamped, and community-identified wildlife observations. Learning to programmatically interact with such APIs is a fundamental skill for any data scientist, as most real-world projects begin with data acquisition from external sources.
2. **Transfer Learning:** Instead of training a deep neural network from scratch (which requires massive datasets and computational power), you will use transfer learning. This technique involves taking a model pre-trained on a large dataset (like ImageNet) and adapting it to a new, specific task. It's a highly effective and resource-efficient approach for most computer vision problems.
3. **MobileNetV3:** You will use MobileNetV3, a state-of-the-art convolutional neural network (CNN) designed for efficiency. Its architecture is optimized for high accuracy on mobile and embedded devices with limited computational resources. This makes it a perfect choice for this task, where you are constrained to training on a CPU. Understanding how to choose a model that fits your hardware constraints is a key practical skill.
4. **PyTorch:** You will implement your pipeline using PyTorch, one of the leading deep learning frameworks. Its flexibility and intuitive, Pythonic interface make it ideal for both research and production. You will use it to load the pre-trained model, define your data loaders, write the training loop, and implement the evaluation logic.

5. **Grad-CAM (Gradient-weighted Class Activation Mapping)**: A model that achieves high accuracy is useful, but a model you can understand is even better. Grad-CAM is a powerful XAI technique that produces visual explanations for a CNN's predictions. It highlights the regions in an image that were most important for a particular classification decision. This helps in debugging the model, verifying that it's learning relevant features (e.g., a bird's beak vs. a tree branch in the background), and building trust with end-users.

Project Architecture and Workflow

Your project will follow a standard machine learning pipeline structure. Each stage will be implemented as a separate, executable script, promoting modularity and reproducibility. The entire workflow can be visualized as follows:



This diagram shows the flow from data acquisition to final model interpretation. Each box represents a component or data artifact you will create. Your scripts will orchestrate the transitions between these stages.

Step-by-Step Implementation Guide

Follow these steps to build your project. Each step corresponds to a script you will write.

1. Environment Setup

Before writing any code, you must create a reproducible environment. You will use Docker for this.

1. **Dockerfile:** Create a `Dockerfile` that starts from a Python base image (e.g., `python:3.9-slim`). It should copy your project files into the container and install all necessary Python libraries from a `requirements.txt` file. Key libraries will include `torch`, `torchvision`, `numpy`, `pandas`, `matplotlib`, `pillow`, and `requests`.
2. **docker-compose.yml:** Create a `docker-compose.yml` file to define your application service. This makes it easy to build the Docker image and run commands within the container. For instance, you can define a service named `app` that builds from your Dockerfile and mounts your local code directory as a volume for easy development.
3. **Project Structure:** Organize your repository with a clear directory structure. A good starting point is:

```
├── data/
│   ├── raw/
│   └── processed/
├── models/
├── notebooks/      # Optional, for experimentation
├── results/
│   └── grad_cam_outputs/
├── scripts/
├── .env.example
├── docker-compose.yml
├── Dockerfile
├── README.md
└── requirements.txt
```

2. Data Acquisition (`scripts/01_fetch_data.py`)

This script will be responsible for downloading the image data.

1. **Goal:** Programmatically download images for at least 10 different species within a single taxonomic order from the iNaturalist API.
2. **Why:** Automating data collection is the first step in any scalable ML pipeline. Using an API allows you to refresh or expand your dataset easily.
3. **What to Do:**
 - i. Choose a taxonomic order with plenty of observations (e.g., Lepidoptera - butterflies and moths, taxon ID: 47157). You can find taxon IDs by searching on the iNaturalist website.
 - ii. Use the `GET /observations` endpoint of the iNaturalist API. You will need to query for observations that are `research` grade, have photos, and belong to your chosen taxon.
 - iii. Iterate through different species within that order until you find 10 species that each have at least 200 research-grade observations with images.
 - iv. For each observation, extract the image URL (prefer `large_url` or `medium_url`).
 - v. Download each image and save it to a structured directory: `data/raw/{species_name}/{observation_id}.jpg` . Use the scientific name for the species name to avoid ambiguity.
 - vi. Make your script robust to API errors and consider adding delays between requests to respect API rate limits.

3. Data Preprocessing (`scripts/02_preprocess_data.py`)

This script prepares the raw data for model training.

1. **Goal:** Split the downloaded images into training, validation, and test sets, and create a class mapping.
2. **Why:** A proper data split is essential for unbiased model evaluation. The validation set is used for hyperparameter tuning, and the test set is used for the final, one-time performance assessment.
3. **What to Do:**
 - i. Scan the `data/raw/` directory to get a list of all images and their corresponding species labels.
 - ii. Perform a stratified split to ensure that the proportion of each species is roughly the same across the train, validation, and test sets. A common split ratio is 70% train, 15% validation, and 15% test.
 - iii. Copy the image files into a new directory structure: `data/processed/{split_name}/{species_name}/` , where `split_name` is one of `train` , `val` , or `test` .
 - iv. Create and save a JSON file `data/processed/class_map.json` that maps each species name (string) to a unique integer index (e.g., `{"Papilio`

`polyxenes": 0, "Danaus plexippus": 1, ...} ). This is necessary for the model's loss function.`

4. Model Training (`scripts/03_train_model.py`)

This is the core script where you will fine-tune your model.

1. **Goal:** Fine-tune a pre-trained MobileNetV3 model on your custom dataset.
2. **Why:** Fine-tuning allows you to leverage the powerful features learned from the large ImageNet dataset, leading to much better performance than training from scratch, especially with a smaller dataset.
3. **What to Do:**
 - i. Use `torchvision.models` to load a pre-trained `mobilenet_v3_small` model.
 - ii. The pre-trained model was trained on ImageNet (1000 classes). You must replace its final fully-connected layer (the classifier) with a new one that has an output size matching the number of species in your dataset (10).
 - iii. Create PyTorch `Dataset` and `DataLoader` objects for your train and validation sets. The `DataLoader` for the training set should shuffle the data. Implement data augmentation for the training set (e.g., random horizontal flips, color jitter) to improve model robustness.
 - iv. All images must be resized and normalized to match the input requirements of MobileNetV3.
 - v. Implement the training loop. For each epoch, iterate through the training data, compute the model's output, calculate the loss (using `CrossEntropyLoss`), perform backpropagation, and update the model's weights (using an optimizer like `Adam`).
 - vi. After each epoch, evaluate the model on the validation set to monitor for overfitting. Track training loss, validation loss, and validation accuracy.
 - vii. Save the training progress (epoch, losses, accuracy) to `results/training_log.csv` .
 - viii. After training is complete (e.g., after a fixed number of epochs or when validation accuracy plateaus), save the weights of your best-performing model to `models/species_classifier.pth` .

5. Model Evaluation (`scripts/04_evaluate_model.py`)

This script will assess the final performance of your trained model on the unseen test set.

1. **Goal:** Calculate key performance metrics and generate a confusion matrix.

2. **Why:** This provides an objective measure of how well your model will generalize to new data. The confusion matrix helps you understand which species the model is struggling with.
3. **What to Do:**
 - i. Load your saved model from `models/species_classifier.pth`.
 - ii. Create a `DataLoader` for the test set. No data augmentation should be used here.
 - iii. Run the model in evaluation mode (`model.eval()`) over all images in the test set.
 - iv. Calculate the overall top-1 accuracy.
 - v. Calculate macro-averaged precision, recall, and F1-score to get a sense of performance across all classes, balanced.
 - vi. Save these metrics to `results/evaluation_metrics.json`.
 - vii. Generate a confusion matrix showing the true labels vs. the predicted labels. Use a library like `matplotlib` or `seaborn` to visualize this matrix and save it as `results/confusion_matrix.png`. Also, save the raw matrix data to `results/confusion_matrix.csv`.

6. Model Interpretation (`scripts/05_run_grad_cam.py`)

This script will generate visual explanations for your model's predictions.

1. **Goal:** Implement Grad-CAM to highlight the image regions your model focuses on when making a prediction.
2. **Why:** This step moves beyond *what* the model predicts to *why*. It's crucial for debugging, building trust, and ensuring the model has learned meaningful patterns.
3. **What to Do:**
 - i. Load your trained model and select a few sample images from your test set (e.g., one or two per class).
 - ii. To implement Grad-CAM, you need to get the gradients of the score for the target class with respect to the feature maps of the final convolutional layer of your model.
 - iii. The gradients are then global-average-pooled to get neuron importance weights.
 - iv. You compute a weighted combination of the forward-pass feature maps and follow it with a ReLU to get the final heatmap. The heatmap shows which areas were important for the prediction.
 - v. Overlay this heatmap (with transparency) on top of the original input image. Save the resulting visualizations to the `results/grad_cam_outputs/` directory.

7. Performance Analysis (`results/performance_analysis.md`)

Finally, document your findings.

1. **Goal:** Analyze and report on the model's performance and limitations.
2. **Why:** Communicating results is a critical skill for a data scientist. This report demonstrates your ability to interpret model performance and think critically about next steps.
3. **What to Do:**
 - i. Create a Markdown file named `performance_analysis.md` inside the `results/` directory.
 - ii. Report the final top-1 accuracy on the test set.
 - iii. Analyze the confusion matrix. Are there any pairs of species that are frequently confused? Why might that be (e.g., they are visually similar)?
 - iv. Crucially, if your model's accuracy is below 80%, include a dedicated section titled "Architectural Changes for Improvement." In this section, explain what changes would be necessary to reach the target. This could include using a larger model backbone (e.g., ResNet50), training for more epochs, collecting more data, or leveraging GPU resources for faster experimentation.

FAQ

1. Q: The iNaturalist API is slow or I'm getting rate-limited. What should I do?

1. A: The API has a default limit of 60 requests per minute. You can make up to 10,000 requests per day. Ensure your data acquisition script includes a delay (e.g., `time.sleep(1)`) between requests to stay within the limit. For faster development, you can start by downloading a smaller dataset (e.g., 5 species with 50 images each) and scale up once your pipeline is working. More information is in the [iNaturalist API Recommended Practices](#).

2. Q: Training is taking too long on my CPU. Is that expected?

1. A: Yes, training a deep learning model, even a small one like MobileNetV3, can be slow on a CPU. A full training run might take several hours. This is a deliberate constraint of the task to make you appreciate the importance of model efficiency. For initial debugging, train for only 1-2 epochs on a small subset of the data to ensure your code runs without errors.

3. Q: My model's accuracy is much lower than 80%. What am I doing wrong?

1. A: There could be several reasons. First, double-check your data preprocessing pipeline—are you normalizing images correctly? Second, review your training loop—is the learning rate appropriate? A learning rate that is too high can prevent convergence, while one that is too low can result in slow training. Third, consider the data itself—some species may be inherently very difficult to distinguish. This is where your `performance_analysis.md` report becomes important. Documenting these challenges is a key part of the task.

4. Q: How do I find the final convolutional layer for Grad-CAM in MobileNetV3?

1. A: You can inspect the model architecture by printing `model` in PyTorch. For `mobilenet_v3_small`, the target layer is typically within the `features` attribute, often the last convolutional block. You can access it via `model.features`. You'll need to register a hook to the layer to capture its feature maps and gradients during the forward and backward passes. See the official [PyTorch documentation on hooks](#).

Core Requirements

1. The project must be fully containerized using Docker.

Contract:

1. A `Dockerfile` must be present in the root directory to define the application image.
 2. A `docker-compose.yml` file must be present in the root directory.
 3. Running `docker-compose up --build` must successfully build the Docker image without errors.
 4. Running `docker-compose run app bash` must start a container and provide a bash shell within the configured environment.
-

2. An example environment file must be provided to document necessary configurations.

Contract:

1. A file named `.env.example` must exist in the root directory.
 2. It must document any environment variables needed to run your scripts, such as the `TAXON_ID` for iNaturalist or the number of species to download.
 3. It should use placeholder values, not real secrets. For example:
`TAXON_ID=47157` .
-

3. A script must download image data from the iNaturalist API and organize it correctly.

Contract:

1. A script must exist at `scripts/01_fetch_data.py` .
 2. When executed, this script must create a directory structure under `data/raw/` .
 3. The `data/raw/` directory must contain at least 10 subdirectories, where each subdirectory name is a species' scientific name.
 4. Each of these species subdirectories must contain at least 200 image files (e.g., `.jpg`).
-

4. A script must preprocess the raw data, splitting it into training, validation, and test sets.

Contract:

1. A script must exist at `scripts/02_preprocess_data.py` .
 2. When executed, it must create three subdirectories inside `data/processed/` : `train` , `val` , and `test` .
 3. Each of these split directories must contain subdirectories for all downloaded species.
 4. The data must be split with approximately a 70% train, 15% validation, and 15% test ratio. The total number of images across all splits must equal the number of raw images.
-

5. The data processing script must generate a JSON file mapping class names to integer labels.

Contract:

1. The script `scripts/02_preprocess_data.py` must generate a file at `data/processed/class_map.json`.
2. This file must be a valid JSON object.
3. The keys of the JSON object must be the string names of the species, and the values must be unique integers (0, 1, 2, ...).

-
6. A script must train the image classification model and save the final trained model artifact.

Contract:

1. A script must exist at `scripts/03_train_model.py`.
2. The script must use a pre-trained `mobilenet_v3_small` from `torchvision`.
3. Upon successful execution, it must save the trained model's state dictionary to a file at `models/species_classifier.pth`.

-
7. The training script must log performance metrics for each epoch to a CSV file.

Contract:

1. The script `scripts/03_train_model.py` must generate a file at `results/training_log.csv`.
2. The CSV file must contain the following columns: `epoch`, `train_loss`, `val_loss`, `val_accuracy`.
3. There should be one row for each epoch of training.

-
8. A script must evaluate the trained model on the test set and save the summary metrics.

Contract:

1. A script must exist at `scripts/04_evaluate_model.py`.
2. When executed, it must load the model from `models/species_classifier.pth` and generate a file at `results/evaluation_metrics.json`.

9. The JSON file with evaluation metrics must adhere to a specific schema.

Contract:

1. The file `results/evaluation_metrics.json` must be a valid JSON object.
2. It must contain the following keys, with floating-point number values:
 1. `overall_accuracy`
 2. `macro_precision`
 3. `macro_recall`
 4. `macro_f1_score`

```
{  
  "overall_accuracy": 0.85,  
  "macro_precision": 0.84,  
  "macro_recall": 0.85,  
  "macro_f1_score": 0.845  
}
```

-
10. The evaluation script must generate both a visualization and a data file for the confusion matrix.

Contract:

1. The script `scripts/04_evaluate_model.py` must generate two files:
 - i. `results/confusion_matrix.png` : An image file visualizing the confusion matrix.
 - ii. `results/confusion_matrix.csv` : A CSV file containing the raw data of the confusion matrix. If there are N species, this file should have N rows and N columns, with headers indicating the species names.

-
11. A script must generate Grad-CAM visualizations for sample test images.

Contract:

1. A script must exist at `scripts/05_run_grad_cam.py`.
 2. When executed, it must create a directory at `results/grad_cam_outputs/`.
 3. This directory must be populated with at least 5 `.png` image files, each showing a Grad-CAM heatmap overlaid on an original image.
-

12. A markdown report must be created to analyze the model's performance.

Contract:

1. A file must exist at `results/performance_analysis.md`.
 2. The file must be valid Markdown.
 3. It must contain a level-2 heading: `## Final Test Accuracy`.
 4. If the accuracy reported in `evaluation_metrics.json` is less than 0.8, the report must also contain a level-2 heading: `## Architectural Changes for Improvement`.
-

partnr

[About Us](#) [Contact Us](#) [Privacy Policy](#)

[Terms and Conditions](#)

All rights reserved. Copyright, Partnr 2025-26

